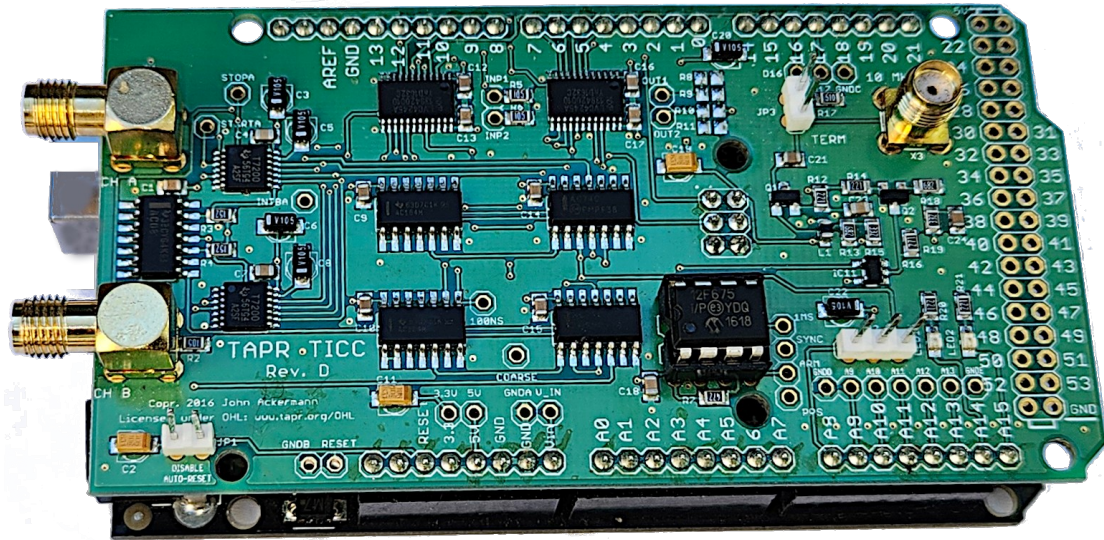


TICC Timestamping Counter Operation Manual

Revised: 15 December 2025

©2016-2025 Tucson Amateur Packet Radio Corporation – CC BY 4.0



Introduction

NOTE: If you want to update the firmware in your TICC to the latest 2025 version, please see Appendix A for instructions.

The TAPR TICC¹ is a two-channel timestamping counter implemented as a shield (daughterboard) for an Arduino Mega 2560 controller. With resolution and jitter of better than 60 picoseconds, it has a one-second Allan Deviation floor of about 5×10^{-11} , which is not much behind today's best commercially available counters.² Better resolution allows measurement of more stable sources at shorter time intervals.

It communicates via USB with a host computer which either logs data to a file or processes it in real time. It can generate time interval and period measurements as well as its native timestamp output.

The TICC can perform over 600 timestamp measurements per second on a single channel. A special binary output mode supports more than 1200 timestamps per second.³

An important note: **the TICC is not a frequency counter.** Rather, it is intended to characterize low frequencies such as pulse-per-second ("PPS") signals.

¹ Totally Integrated Cycle Counter – cute if not precisely accurate!

² References to performance and features in this document are based on the latest 202510xx firmware version which provides better performance and adds features compared to earlier versions.

³ Maximum throughput requires using a 230400 baud serial connection.

What is a Timestamping Counter?

A timestamping counter is a bit different than a traditional electronic counter. Those generally output the time interval between signals arriving on two channels. By contrast, a timestamping counter tags every event on each channel based on its arrival time, using an internal timescale maintained by the counter. The timescale starts when the system boots and can continue indefinitely. A sequence of timestamps from a 1 pulse per second (“PPS”) source might look like this:

```
1.897999794440 chA
2.897999794492 chA
3.897999794549 chA
4.897999794551 chA
5.897999794553 chA
6.897999794552 chA
7.897999794667 chA
```

Since the pulses occur nominally once per second, the timestamps increase at that rate. But not *exactly* at that rate – no two clocks ever keep precisely the same time, and all have some amount of jitter or noise in their readings. From the slight differences revealed by the timestamps one can determine frequency offset (difference between measured and nominal event rate) and stability (noise from reading to reading). Thus, a timestamping counter can be used to characterize clock performance. The TICC has two measurement inputs and can measure one or two independent sources simultaneously, generating timestamps for each.

Timestamps form a basic building block and can be used to generate other output formats such as time interval – the time between an event on channel 0 and the corresponding event on channel 1.⁴ An example of time interval measurement is putting a PPS signal from the device under test (“DUT”) on `ch0` and the PPS signal from a GPS receiver on `ch1`. The changes from second to second can be used to determine the DUT’s frequency and stability relative to the GPS system. Time interval is a more traditional measurement technique, but timestamps offer advantages, not least of which is the ability to measure two independent sources simultaneously.

Additional TICC modes are `Period`, which reports the time between an event on a channel and the last event on that channel; and `3-Corner Hat` which allows a sophisticated measurement technique (see below for more information about that). A `Binary Timestamp` mode allows about double the measurement rate of the normal `Timestamp` mode, but requires the user to process binary data to extract timestamps.

It’s important to realize that every measurement of time or frequency is made in terms of a reference source – it’s a *comparison* of the DUT with the reference. When generating timestamps, the reference is the external 10 MHz source connected to the TICC, called `EXT_REF`. Any drift or noise in `EXT_REF` may hide the performance of the device under test. Ideally, `EXT_REF` should be an order of magnitude more stable and/or accurate than the DUT. If it is not, the timestamps may reflect the `EXT_REF` performance mixed with the DUT, or the `EXT_REF` noise could even dominate the results.

In `Time Interval` mode, the comparison is made between the two input channels (measuring the time between the event on `ch0` and the next event on `ch1`), not between the input and `EXT_REF`. The `EXT_REF` signal is still required for the TICC to work, but its stability and accuracy matter only during the usually very short interval between `ch0` and `ch1` pulses – in this case `EXT_REF` serves as a “transfer standard” rather than absolute reference. Thus, for time interval measurements its quality is less important than in timestamp mode, and often a modest oscillator will suffice.

4 In this document, channel 0 or “`ch0`” and channel 1 or “`ch1`” are used to refer to the two hardware inputs. In the Arduino software, the channels are assigned names, by default “`chA`” and “`chB`”.

The TICC normally outputs data in ASCII serial format via USB which can be logged to a file using a standard terminal program. The output files can be read by standard analysis software such as John Miles' TimeLab⁵ and W. J. Riley's Stable32.⁶ TimeLab can also read live TICC data via USB serial connection. For high measurement rates, a binary timestamp output mode is available. Sample programs for logging and decoding the binary output are provided in the Github repository.⁷

Connection and Configuration

Hardware

The TICC circuit board is a shield that mounts to an Arduino Mega 2560 processor. The Arduino runs open source firmware which is available at <https://github.com/TAPR/TICC>. TICCs provided by TAPR have the board and processor mated, and firmware loaded.

The TICC obtains its power from the Arduino, using either the USB connector or the DC input connector (7 to 12 volts). Its current drain is typically well under 100 milliamps. Tests have shown that there is no performance improvement in using the external power connector instead of USB power.

External Reference

Connect a 10 MHz reference source to the vertical SMA receptacle on the board. This is the source of EXT_REF. Signal levels from -10 to +13 dBm will work. Remember that in Timestamp mode the 10 MHz source is the reference, so its performance sets the performance limit of the system.

IMPORTANT: A two pin header on the TICC board near the EXT_REF input connector can be shorted to provide a 50 ohm termination for the external reference source. THIS IS STRONGLY RECOMMENDED as without termination reflections on the coaxial cable can cause ringing, particularly if the source is a square rather than sine wave, and this can cause clocking errors. If you see odd results, such as timestamps increasing faster than they should, check the termination.⁸

Signal Inputs

The `ch0` and `ch1` SMA connectors accept digital signal levels of up to 5 volts without damage. The trigger level is about 1.7 volts. The input impedance is 1 megohm.

If signal levels allow, it is recommended to use a 50 ohm in-line or "through" terminator connected at the TICC SMA input. This will provide a match to coax cable and help reduce ringing that could cause false triggering. However, SMA through terminators are not easy to find. A 6 dB attenuator at the SMA connector can do the job, providing about as much voltage to the TICC input as a through termination, and providing a decent match (about 1.7 VSWR) to minimize ringing.

Other Connections

By default, the Arduino restarts every time a serial connection is made to it. In many cases this is fine, but there are times where this behavior is inconvenient. There is a two-pin header at the lower left corner of the TICC board (with the connectors facing left) labeled `DISABLE AUTO-`

5 <https://www.miles.io/timelab/beta.htm>

6 (note that this is an HTTP, not HTTPS page, but it is safe when last checked.)

7 , in the `bin/` directory.

8 I am emphasizing this because it just bit me during testing. I was seeing timestamps advance by two seconds per second. This was because the 10 MHz source was an HCMOS (square wave) output and I had removed the termination jumper. The cable ringing was causing a double pulse which made the TICC counters run at twice the correct rate.

`RESET`. Shorting it will prevent this auto-restart behavior. **If you install this jumper, remove it before attempting to update the TICC firmware.**

Next to the reset-inhibit header are two pins labeled `RESET` and `GROUND`. Momentarily shorting those pins will force a restart.

There are two LEDs on the lower right corner of the board that indicate activity on each of the two input channels. Note that these are not directly driven by the TICC inputs, but are triggered by the software processing loop. They provide an indication that processing is occurring but cannot be used to estimate the duty cycle of the input pulse. If you want to mirror these LEDs with external ones, they can be connected at Arduino pins `A12` and `A13` which go high when the LED is active.

Arduino pin `A11` can drive an LED to provide a visual indication of the presence of the 10 MHz reference clock. To be more precise, it is illuminated by a watchdog that checks to see whether the `PICcount` variable is being incremented as it should be. Loss of reference will cause the LED to extinguish (pin `A11` goes low).

The three-pin header next to the on-board LEDs is intended for use when multiple TICCs are tied together in a single measurement system with one “primary” unit that provides coarse clock and synchronization to the other(s). This header provides the `COARSE_CLOCK` signal on pin 3 and a synchronizing pulse at startup on pin 1. Pin 2 is ground. For more details on multiple synchronized TICCs, refer to the TAPR Multi-TICC application note.

The TICC communicates via USB with serial port emulation. The port parameters are 115200, 8N1. Output lines are terminated with carriage return and linefeed. All data is 7-bit ASCII with no special characters sent. In the 2025 firmware, the baud rate can be set from the configuration menu. Changing the speed to 230400 baud is required for maximum measurement rate.

It is possible to use direct serial communications by connecting to Arduino pins 0 (`RX`) and 1 (`TX`). These are 5V TTL pins, so you will need a level converter to drive a “true” RS232 port. The Arduino USB to UART converter also feeds these pins, so if you are using direct serial connection, make sure nothing is connected to the USB connector.

Startup

At startup the TICC outputs a splash screen like the one below and then pauses for about five seconds. Entering any character during that time will bring up the configuration menu.

```
# TAPR TICC Timestamping Counter
# Copyright 2016-2025 N8UR, K9TRV, NH6Z, WA8YWQ
#
# #####
# TICC Configuration:
# Software Version: 20251101.1 (RELEASE)
# EEPROM Version: 13, Board Version: D
# Board Serial Number: 33943FD8
# Measurement Mode: Timestamp
# Timestamp Wrap: 0 (no wrap)
# Output Decimal Places: 11
# SyncMode: P
# Serial Baud Rate: 115200
# Channel Names: A/B (C)
# Poll Character: none
# Clock Speed: 10.000000 MHz
# Coarse tick: 100.000000 usec
# Cal Periods: 20
# Propagation Delay ps: 0 (ch0), 0 (ch1)
# Timeout: 0x05
# Time Dilation ps: 2500 (ch0), 2500 (ch1)
# fixedTime2 ps: 0 (ch0), 0 (ch1)
# FUDGE0 ps: 0 (ch0), 0 (ch1)
# #####
#
# Type any character for config menu
# .....
#
# timestamp (seconds with 11 decimal places)
0.95009165963 chA
1.08318695576 chB
```

Configuration

The TICC has a number of configurable parameters which are stored in erasable non-volatile memory (“EEPROM”). The parameters can be changed via a set of configuration menus, which are accessible by pressing any key during the startup delay, or at any time by pressing the “#” key.

Command Syntax:

In configuration mode, a #> prompt is displayed indicating that the user may input configuration commands. Commands are terminated with a carriage return.

Each configuration parameter is identified by an ID which is either a single letter in the main menu, or a letter and a single digit in the submenus.

The command parser has three modes:

- **Interactive** – enter an item ID followed by carriage return, and you will be prompted for input.
- **Concise** – enter the item ID and new values on the same line, followed by carriage return. For example, F C/D<enter> to change the channel names to C and D. Spacing between item ID and values is optional; FC/D is also a valid command.
- **Multi-Item** – enter multiple commands on a single line, with semicolons separating them. For example, A1;F C/D;W to change to timestamp mode, change channel names to C and D, write the changes to EEPROM, restart (automatic after EEPROM write).

User-entered input lines are terminated with a carriage return. Configuration parameters are changed by entering the item ID and its new value. Item IDs are one letter for main menu items, and one letter followed by one digit for submenu items.

Item IDs and values may be entered in either lower or upper case. Input is converted to upper case for processing except for the channel name parameter, where a user might want to use lower case letters for the names.

Some parameters take two values, one for each measurement channel. Enter the two values with a “/” separating them. For example, when changing channel names to “C” and “D”, enter F C/D.

When entering the item ID and value together, spaces between them are optional. For example, to change trigger edge, both D F/F and DF/F are valid.

Configuration Settings – Main Menu:

```
# Type any character for config menu
# .....
#
# == TICC Configuration ==
# A - Mode (currently: Timestamp)
# B - Timestamp Wrap digits (currently: 0 - no wrap)
# C - Output Decimal Places (currently: 11)
# D - Primary/Secondary (currently: P)
# E - Channel Names (currently: A/B (C))
# F - Poll Character (currently: none)
# G - Advanced settings
# H - Serial Baud Rate (currently: 115200)
#
# M - Show this menu again
# S - Show startup info
# V - Show firmware version
# W - Write changes to EEPROM (persist across restarts)
#
# 1 - Discard changes and exit
# 2 - Apply changes and resume operation
# 3 - Reset all to defaults and restart
# > █
```

The parameters you can change from this menu are:

- **Mode:** Takes you to the mode submenu.
- **Timestamp Wrap:** If set to 0 (default), timestamps will continue incrementing for about 68 years before overflowing. This can cause loss-of-resolution issues on long runs (see Appendix B for a discussion of this) and for most uses it is recommended to wrap timestamps. “1” means one integer digit before wrap (wrap at 10 seconds), “2” means two digits (wrap at 100 seconds), etc. Timestamp wrapping affects only the `Timestamp` modes and does not affect time `Time Interval` or `Period` modes or the `Binary Timestamp` mode.
- **Output Decimal Places:** Sets the resolution of output data from 0 to 12 places (default is 11 places, or 10 ps).
- **Primary/Secondary:** This sets the synchronization mode. When set to “Primary” (default), the TICC uses its own 100 us `COARSE_CLOCK` signal. When set to “Secondary” the TICC depends on an external source for this as well as a synchronization pulse at startup to align the timescale with that of the primary unit. See the TAPR TICC application note “multi-TICC: A Multi-channel Timestamping Counter Based on the TAPR TICC”⁹ for further details.
- **Channel Names:** Default is A/B but can be changed to other single-character values. This is mainly useful when multiple TICCs are being used together. If you are using 3-Corner Hat mode (see below), you can also set the name for the third channel here (default is C) by including it in the command. e.g., “D/E/F”.
- **Poll character:** Unset by default. If set, the TICC will output data only when the poll character is received, at which time the most recent measurement will be output. Note that other measurements will be lost. Think of this as a trigger function – sending the poll character returns the single most recent measurement.
- **Advanced Settings:** Takes you to the advanced settings submenu.
- **Serial Baud Rate:** Change the console speed. Default is 115200. After changing, make sure you write the change to EEPROM, and then restart with your terminal program set to the new rate.

The “M”, “S”, “V”, and “W” commands allow you to redisplay the menu, show the startup screen with current configuration parameters, show the firmware version and unit serial number, and write changed values to non-volatile memory.

NOTE – if you do not write changes with the W command, they will be lost the next time the TICC restarts.

The “1”, “2”, and “3” commands are self-explanatory.

Note that all output lines generated by the configuration system have “#” characters prepended. This allows them to be treated as comments by TimeLab or other analysis software.

Configuration Settings – Mode Menu

The **Mode** menu allows you to select one of the TICC's measurement modes. To change, enter both characters, e.g., "A1".

```
# > a
#
# -- Mode --
# A1 - Timestamp
# A2 - Paired Timestamp
# A3 - Binary Timestamp
# A4 - Time Interval ch0->ch1
# A5 - Period
# A6 - 3-Cornered Hat
# A7 - Debug
# A8 - Null Output
#
# Current mode: Timestamp
#
# >
```

- **Timestamp:** The default mode which outputs timestamps from channels A and/or B as they are received. Timestamps are always printed in strict ascending order regardless of channel.
- **Paired Timestamp:** When both channels are active, this mode always prints timestamps in pairs with channel 0 then channel 1 order (e.g., `chA` then `chB`). This mode is designed for data to be fed into the TimeLab program, which expects multi-channel data to follow strict pair-order sequence.
- **Binary Timestamp:** Outputs a 12 byte binary message for maximum measurement rate. See below for details. Example Python programs to capture and decode this binary data are available in the repository at <https://github.com/TAPR/TICC/bin/>
- **Time Interval:** Outputs the value of `ch1` minus `ch0` for each pair of measurements.
- **Period:** Outputs the difference between the current and prior values of channels A and/or B, i.e., `ch0 (n)` minus `ch0 (n-1)`.
- **3-Corner Hat:** This is a special mode that outputs the timestamps for channels 0 and 1 as in Timestamp mode, as well as a "synthesized" "`ch2`" timestamp representing the time interval from `ch0` to `ch1`. There's a bit more about this in the System Description section below.
- **Debug:** This mode outputs raw values that are used to calculate timestamps. It's primarily for debugging (duh!).
- **Null Output:** This mode suppresses all data output. It's mainly useful for testing.

Configuration Settings – Advanced Menu

The **Advanced** menu includes settings that are normally only used for special purposes; many of them relate to tweaking calibration. These usually are not required. To change, enter both characters, e.g. G1.

```
# > g
#
# -- Advanced Settings --
# G1 - Clock Speed MHz (currently: 10.000000 MHz)
# G2 - Coarse Tick us (currently: 100.000000 usec)
# G3 - Propagation Delay ps ch0/ch1 (currently: 0/0)
# G4 - Time Dilation ch0/ch1 (currently: 2500/2500)
# G5 - fixedTime2 ps ch0/ch1 (currently: 0/0)
# G6 - FUDGE0 ps ch0/ch1 (currently: 0/0)
#
# >
```

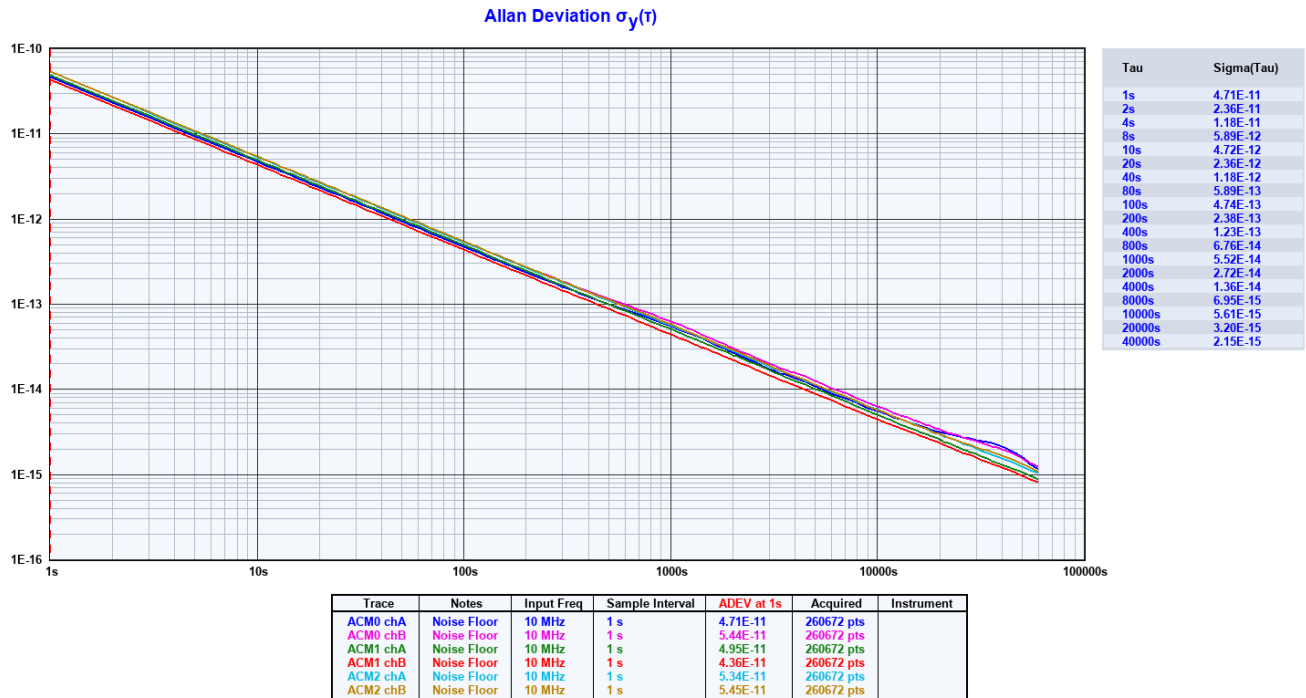
- **Clock Speed and Coarse Tick:** The TICC is normally used with a 10 MHz clock and 10 kHz (100 us) coarse timer. It's possible to change these to use other values with appropriate hardware changes, but **this is untested and unsupported**.

- **Propagation Delay:** This adds a user-specified delay that allows compensation for cable or other delays. Each channel can be set individually.
- **Time Dilation:** This is a magic number that was experimentally determined to deal with a small non-linearity in the TDC over different time-of-flight periods.
- **FixedTime2:** This can be used to override a calculated value in the time of flight calculation. It should normally be set to zero; it probably serves no real purpose other than for experimentation. There is a note in the Github TAPR/TICC/docs directory that provides some information about this.
- **FUDGE0:** This is another knob to add delays to one or both channels. Its delay is additive to the Propagation Delay if set.

TYPICAL PERFORMANCE

Resolution and Jitter

The following Allan Deviation plot show a test of three TICC's running firmware version 20251101.1. All signals were derived from a single 10 MHz source. Using a common source for all inputs eliminates external noise, and allows the performance of the TICC itself to be measured.



Test Configuration Details

A single 10 MHz source from an HP 5065A Rubidium standard drove both the TICC `EXT_REF` input and a TAPR TADD-2¹⁰ pulse-per-second divider. The `EXT_REF` termination jumper on the TICC was installed to provide a 50 ohm load.

The TADD-2 output outputs were connected to the `ch0` and `ch1` inputs on each TICC, with a 6 dB attenuator connected directly at the input to act as a feed-through termination. The signal at the TICC inputs was about 2.5 volts peak-to-peak. The output was logged to a file and then imported to TimeLab for processing.

Measurement Rate

Changes made in the 202511xx.x firmware version have significantly improved the maximum measurement rate. To test this, a function generator was used to generate square waves with 5 volt peak amplitude and the TICC's output was monitored with the `ticc_rate_test.py` program (available in the `bin/` folder of the TICC github repository). The TICC was placed in `Timestamp` mode and there was a 6 dB attenuator at the channel inputs.

¹⁰ <https://tapr.org/product/tadd-2-pulse-per-second-divider/>

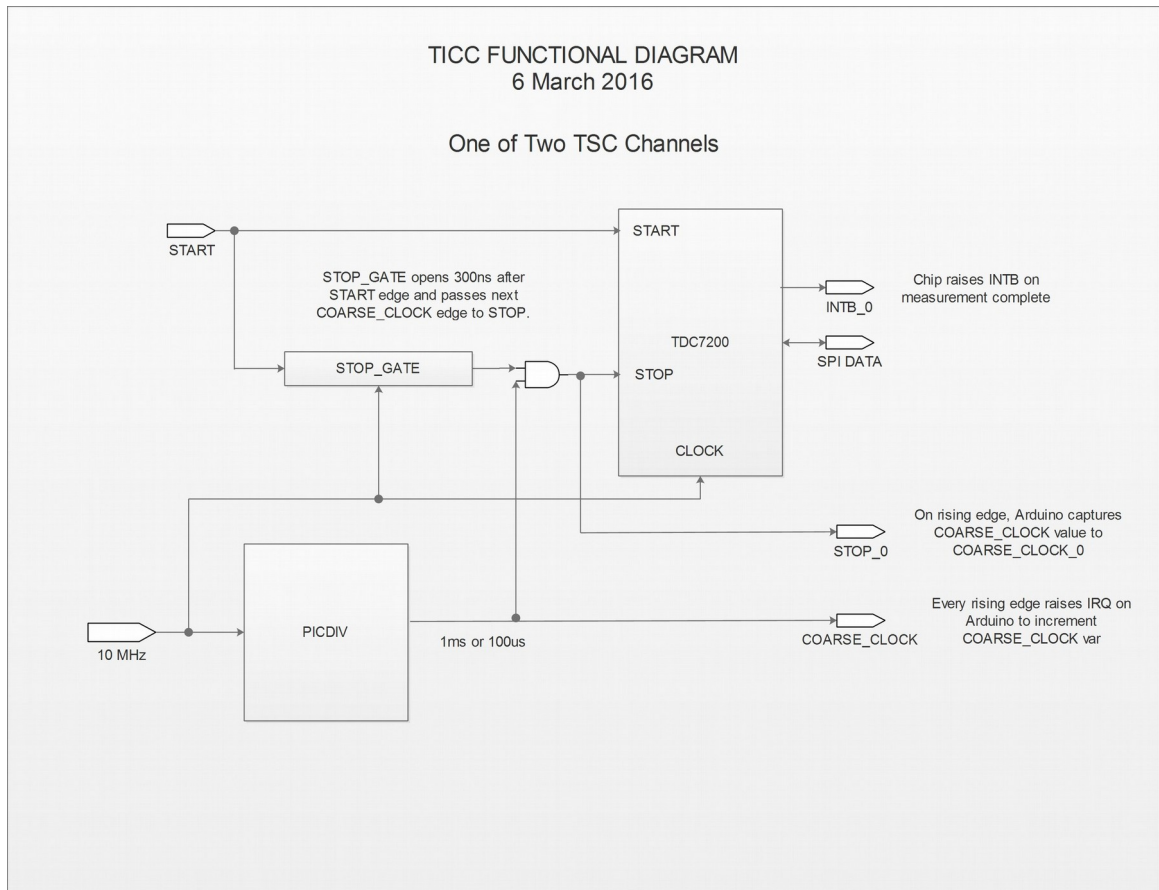
When driving a single channel, the TICC was able to generate timestamps at a rate of about 585 measurements per second at 230400 baud. In Binary Timestamp mode, about 1130 measurements/second are possible at 230400 baud.

SYSTEM DESCRIPTION

The core characteristic of an electronic counter is its resolution – how precisely it can measure and display time. A limitation of digital counters is that their resolution is tied to the clock speed. If the clock only ticks once per millisecond, you cannot measure with more resolution than that. A clock rate of 1 GHz, which is challenging to achieve, provides a resolution of one nanosecond. In order to obtain higher resolution, special techniques such as interpolation between clock cycles are required. Traditionally, this has required analog circuits using methods such as measuring the decreasing voltage over time across the terminals of a capacitor. The best of these schemes can yield resolution and jitter of around 20 picoseconds, but they are complex and require periodic calibration.

The TICC uses a different method to measure sub-clock-cycle times, thanks to the Texas Instruments TDC7200¹¹ time-to-data converter chip which uses digital techniques to achieve picosecond resolution. The TICC adds additional logic in both hardware and software to a pair of TDC chips to create a complete two channel timestamping counter. This technique yields a resolution of about 55 picoseconds.

It's difficult to visualize how the TICC hardware and software work together by viewing either in isolation. Hopefully the overview that follows provides a picture of how the pieces work together. As you read it, keep this block diagram of one TICC channel in mind:



Each time a pulse from the DUT appears on a TICC input channel, the TDC7200 for that channel measures the time interval from the leading edge of that pulse until the leading edge of the next tick of `COARSE_CLOCK`.

¹¹ Often abbreviated in this document as “TDC”

An external 10 MHz source (EXT_REF) drives the CLOCK input of the TDC7200 chips. COARSE_CLOCK is a 10 kHz (100 us) signal that is derived from EXT_REF by a PIC-based divider.¹² Thus the two primary clocks in the TICC are synchronous with each other. Each tick of COARSE_CLOCK also triggers a hardware interrupt that increments the Arduino variable PICcount, which represents the time since TICC power-up with 100 microsecond resolution. PICcount is a 64 bit integer that will take about 30 billion years to overflow, so don't lose sleep over it.

How the TDC7200 Works

The TDC7200 measures the elapsed time (called in the datasheet “time of flight” or “TOF”) from the occurrence of a pulse on its START pin until a pulse on its STOP pin. It does this with two clocks. The first is EXT_REF, providing 100 ns resolution. The second is an internal ring oscillator that operates at a multi-gigaHertz rate.

A ring oscillator is made of an odd number of digital inverters hooked input-to-output – sometimes described as “a snake eating its own tail.” Feedback from the end of the chain to the beginning produces a free running oscillator with a frequency based on the propagation delay of the gates. The 63 gates in the TDC ring run at nearly 17 GHz, or a period of about 60 picoseconds. In this document, the ring oscillator is called FINE_CLOCK.

The diagram below¹³ shows how the TDC7200 generates the time-of-flight. When triggered by a START signal, it counts FINE_CLOCK ticks (TIME1 in the diagram) until the next EXT_REF clock arrives, when it switches to counting 100 ns EXT_REF cycles (CLOCK_COUNT1) until the STOP signal arrives. Finally, the STOP signal starts another FINE_CLOCK counter until the next EXT_REF clock edge (TIME2).

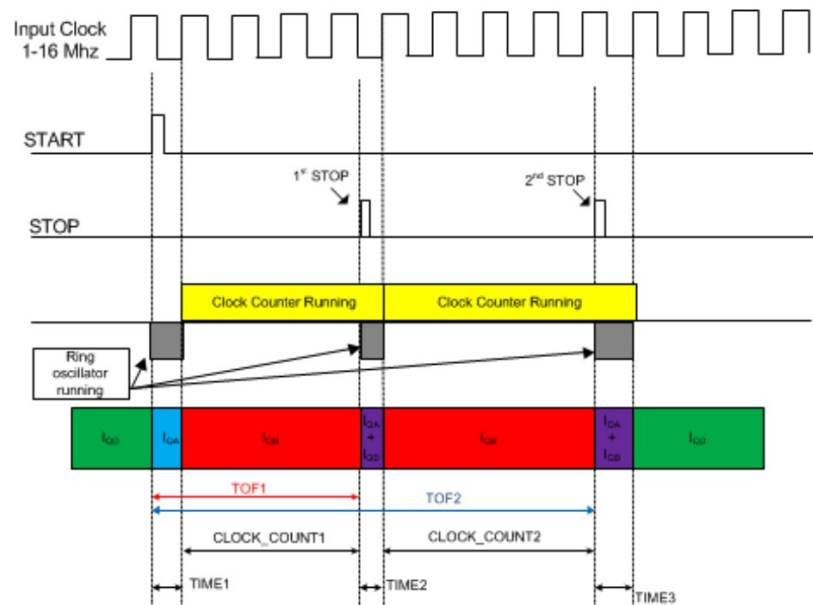


Figure 18. Measurement Mode 2

The time of flight is calculated as:

$$(\text{TIME1_COUNT} * \text{RING_PERIOD}) + (\text{CLOCK_COUNT1} * 100 \text{ ns}) - (\text{TIME2_COUNT} * \text{RING_PERIOD})$$

(In some applications, multiple STOP signals may be received after a single START signal, so the illustration above shows additional measurements. The TICC uses only the first STOP pulse.)

¹² The PIC runs firmware called “PD15” developed by Tom Van Baak and used with permission. Tom has developed several flavors of this firmware. Check his web page at [http://www.ticcd.com](#) for more information.

¹³ Diagram is from the TDC7200 datasheet.

The ring oscillator is very fast and thus provides great resolution, but it is free-running and not stable. Factors such as temperature and supply voltage affect its frequency. One of the TDC7200's really clever features is the calibration cycle that occurs at the end of every measurement. This measures the ring oscillator speed relative to the external reference frequency. The calibration counts the number of ring cycles occurring during a fixed number (default 20) of 100 ns reference clock ticks and thus derives the actual speed of the ring oscillator. This calibration addresses the instability problem.

When it finishes a measurement, the TDC raises a hardware interrupt line (`IRQB`) which causes the Arduino software to request the TDC to deliver the raw `TIME1_COUNT`, `CLOCK_COUNT1`, `TIME2_COUNT`, and calibration results via the SPI interface. The Arduino uses that data to calculate the actual time of flight.

The TDC7200 has two limitations, though, that prevent it from being a useful on its own as a timestamping counter: The first is that the maximum time it can measure in the mode we use is about 6 milliseconds. That limitation is addressed by the `PICcount` variable that extends the range almost indefinitely as discussed below. The second limitation is that the time from START to STOP must be greater than 200 nanoseconds.¹⁴

The TDC measures the time from the START pulse until the next tick of the 100 us `COARSE_CLOCK`, which is applied to the TDC STOP pin. However, that tick might arrive less than 200 ns after the START input, violating the TDC7200's timing requirements. To prevent that, the TICC has a circuit that gates `COARSE_CLOCK` to ensure the minimum START to STOP time requirement is met.

This “stop gate” is clocked by `EXT_REF` and blocks `COARSE_CLOCK` from reaching the TDC's STOP input for 3 `EXT_REF` clock cycles after the START pulse. After the gate opens, the next `COARSE_CLOCK` tick will stop the TDC timer. As a result, the time of flight will range from a minimum of just over 200 ns (`COARSE_CLOCK` arrives just as the stop gate opens) to 100.299...us (`COARSE_CLOCK` arrived just under 300 ns after START, so wait until the next `COARSE_CLOCK` tick). This range fits comfortably within both the minimum and maximum allowed time of flight.

The gated `COARSE_CLOCK` tick that stops the TDC also triggers an Arduino hardware interrupt, capturing the current `PICcount` value into the `PICstop` variable for that channel. `PICstop` thus represents the coarse part of the timestamp (100 us resolution) for that event. To be more precise, it represents the coarse value immediately after the event, not before it – that is important to the calculation that follows.

Processing in the Arduino

NOTE: *The following discussion is based on the latest 2025 firmware version. Earlier versions performed the same functions but the code was significantly rewritten in 2025.*

When the TDC finishes its measurement, it sends a hardware interrupt (`INTB`) to the Arduino. That causes the Arduino to query the TDC via the SPI communication bus for the registers containing the raw data values which the Arduino uses to calculate the time of flight. The TDC read, write, and time of flight calculation routines are all in the source file `tdc7200.cpp`. The calculated time of flight as well as other information used to calculate the timestamp are placed in a variable structure called `channels[i]` where `i` is the channel number (0 or 1).

To calculate the timestamp, the main loop in `TICC.ino` does:

```
timestamp = (PICstop * PICTICK_PS) - tof
```

¹⁴ In measurement mode 2, which is what the TICC uses. Measurement mode 1 only requires 12 nanoseconds, but is limited to a maximum time interval of about 500 nanoseconds, which is not enough for the TICC's needs.

where `PICTICK_PS` is the number of picoseconds per `COARSE_TICK` – 100,000,000.

Note that `tof` is subtracted from the coarse timestamp to yield the final timestamp – because `PICstop` occurred after the event, not before it.

That's not the way we actually generate the timestamp, though, because 64 bit operations like multiplying `PICstop` by `PICTICK_PS` are very slow on the Arduino. To avoid and maximize throughput, an accumulator scheme is used where the difference between the current and prior `PICstop` values is taken. This is almost always small enough to fit into a 32 bit variable, so it allows more efficient subsequent processing. The comments in `TICC.ino` try to explain this in more detail.

The TOF is calculated to single picosecond resolution (12 decimal places) in a 64 bit variable. But for efficiency, because 64 bit support in the Arduino C libraries is limited and slow, 32 bit variables are used wherever possible, and all math is done with integers to avoid floating-point precision issues. Timestamps are stored in the `Timestamp64` struct with two elements: 32 bit integer for seconds, and 64 bit unsigned integer for picoseconds. Helper functions are used to manipulate `Timestamp64` variables and to print them. Using a 32 bit integer for seconds, the timestamp value will overflow in about 68 years.

While the timestamp is calculated with single picosecond precision, by default only 11 decimal places are output (10 picosecond resolution). Since the actual resolution is about 60 picoseconds, showing the 12th decimal place does not provide any benefit. The number of displayed digits can be changed in the configuration menu.

Print formatting in the 8-bit Arduino is particularly slow, and the native `printf()` family functions do not handle 64 bit variables, or do floating point formatting. Nor can the `Serial.print()` functions print 64 bit variables. To work around this, specialized print functions `print_timestamp()` and `print_int64()` are used. These are by far the fastest print routines found out of several tried. They come close to saturating the performance of a 115200 baud serial link.

For maximum throughput, a binary timestamp mode moves some processing overhead to the host computer and outputs a 12 byte binary message. That message includes two header bytes, a single byte channel ID, the lowest four bytes of the `PICstop` variable, and the four bytes of the `tof` variable. The host is responsible for performing the calculation:

$$ts_picos = (PICstop * PICTICK_PS) - tof$$

(where `PICTICK_PS` is 10,000,000 for the default configuration) and for handling `PICstop` wraps, which occur at about 4.92 days. This mode can generate over 1100 single-channel timestamps per second which will saturate a 115200 baud serial stream.

TICC Output Modes

The TICC supports several output modes:

- The default is `Timestamp` mode, which prints the raw timestamp data for each channel. Each data value is followed by the channel ID, e.g., `chA`. Timestamps output for either channel and removal of one input will not stop output of the other. The software tries to pair `ch0` and `ch1` samples and outputs them by default in strict ascending timestamp order. However, the TimeLab software expects multi-channel data to be in channel order (e.g., `chA/chB/chA/chB`, not `chA/chA/chB/chB`) and the “paired timestamp” mode does this.
- As noted above `Binary Timestamp` mode allows for about twice the measurement rate as the standard `Timestamp` mode, and also produces more compact output files. But the user is required to convert the binary output into timestamp format.
- In `Time Interval` mode, each pair of `ch0` and `ch1` timestamps is subtracted to yield the `ch0` to `ch1` interval: `ch1 - ch0`, which may be positive or negative.

- In `Period` mode, each output is the result of subtracting the prior timestamp:
 $\text{chX}[n] - \text{chX}[n-1]$.
- `3-Corner Hat` mode is more complicated. 3-cornered hat analysis requires three measurements: $(A - \text{REF})$, $(B - \text{REF})$, and $(B - A)$. In this mode the TICC outputs the timestamp for each channel, and also “synthesizes” a timestamp for the $(B - A)$ difference.

Under some conditions, three cornered-hat analysis can separate out the individual noise contributions of the three sources used in a two-channel timestamp measurement. 3-cornered hat measurements need three. This method only works when the three sources have similar characteristics and the results always need to be reviewed carefully to make sure they make sense. See Bill Riley’s page at <https://www.wriley.com/3-CornHat.htm> for a mathematical analysis.

In `3-Corner Hat` mode, the TICC outputs timestamps for `ch0` and `ch1`, as well as a “synthesized” timestamp `chC` that represents $\text{int}(\text{ch1}) + (\text{ch1} - \text{ch0})$. This is done because TimeLab can’t mix timestamp and time interval data in the same file. Incorporating the seconds part from the `ch1` timestamp (with appropriate borrow/carry logic) generates a timestamp that is mathematically valid and that TimeLab can use.

There is also a `Debug` mode which outputs the raw TDC register values. This is mainly for testing purposes, and a `Null` mode which suppresses all data output, again for testing.

Appendix A

Updating TICC Firmware

The TICC firmware can be updated via USB connection, using a binary image available from <https://github.com/TAPR/TICC/binaries/>. The hex file name indicates the version, e.g., `TICC.ino-202512115.1.hex`.

There are several software tools available to do the upload, for multiple platforms. Included in the <https://github.com/TAPR/TICC/bin/> directory are several programs you can try:

- `AVRDUDESS-2.20-setup.exe` – GUI for Windows (recommended)
- `AVRDUDESS-2.20-portable.zip` – GUI for Linux/Mac (recommended)
- `Xloader.zip` – GUI for Windows; older, very basic program. Use AVRDUDESS instead.
- `ticc_updater.py` – command line tool to update. Good for remote updates. Requires that avrdude be separately installed.

How to Upload TICC Firmware to an Arduino Mega 2560 using AVRDUDESS

- Download and install AVRDUDESS on your system. If on Linux or Mac, you will need to also install mono; according to the AVRDUDESS Github page, for a Debian/Ubuntu system at a minimum you will need `libmono-system-windows-forms4.0-cil` and you might also need to install `avrdude` and `binutils-avr`.
- Download the desired TICC image file, with a `".hex"` extension.
- Connect the TICC via USB.
- Find the TICC's serial or COM port
- In Windows, open Device Manager and expand "Ports (COM & LPT)". Look for "Arduino Mega 2560" (or similar) with a COM number, e.g., COM5. If you see a "USB-SERIAL CH340" device, that's fine too; note its COM number.
- In Linux, look in `/dev` – the Arduino will usually be `/dev/ttyACM0` or `/dev/ttyUSB0`.
- Remember the serial port name (e.g., COM5). You'll select this in AVRDUDESS.
- Start AVRDUDESS. On Windows, if prompted by SmartScreen, choose "More info" → "Run anyway" (if you trust the source). On Linux, you'll need to run under mono and you might need to run as root. From the command line enter: `mono avrduess.exe`.
- Configure AVRDUDESS settings:
 - Programmer: `wiring` (IMPORTANT – not any of the "Arduino" variants)
 - MCU: `ATmega2560`
 - Port: select your COM port (e.g., COM5)
 - Baud rate: `115200`
 - Flash: click the "..." browse button and select your firmware file: `TICC.ino-20250930.1.hex`
 - Under Options, check the box `"-D (disable auto erase)"` (IMPORTANT).

- Upload the firmware: Click “Program”.
- Wait until the log shows messages like:
 - “writing flash ... 100%”
 - “bytes of flash written”
 - “verified” or “avrdude done. Thank you.”
- Do not unplug the board while it’s programming!
- After it finishes the board will reset and run the new firmware automatically. You can close AVRDUDESS.

Troubleshooting

I don’t see a COM port:

- Try a different USB cable/port.
- Reopen Device Manager after unplug/replug.
- If it’s a clone board (CH340), Windows may need the CH340 driver (search “CH340 Windows driver”, install, then replug).

“Programmer is not responding” or “not in sync”:

- Make sure the correct COM port is selected.
- Close any apps using the port (Arduino IDE Serial Monitor, other terminal apps).
- Unplug/replug the USB cable, then try again.
- Double-check Programmer = `wiring`, MCU = `ATmega2560`, Baud = `115200`.

“command failed” during erase:

- Ensure the “-D (disable auto erase)” option is checked.

Appendix B

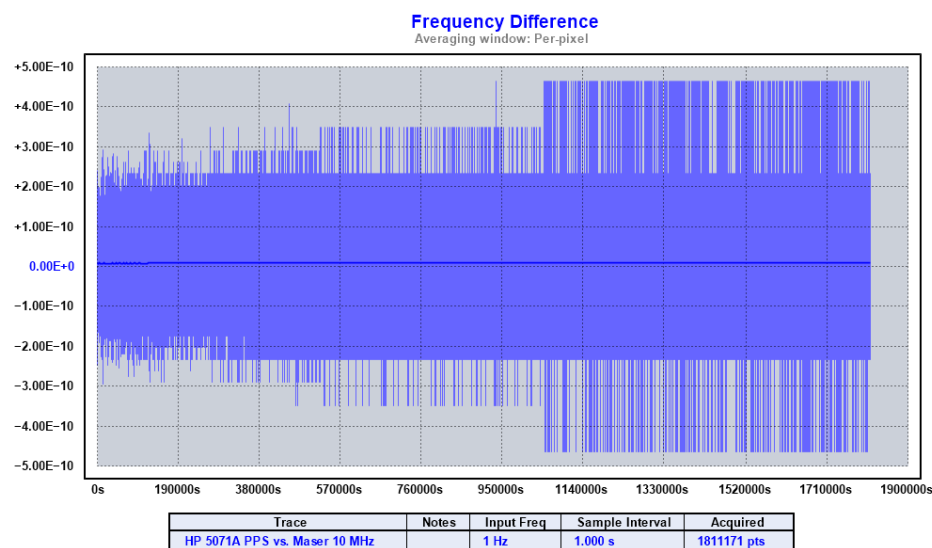
Big Timestamps and Floating Point Precision

TL;DR: Unless you have a good reason not to, wrap timestamps at a reasonable value (e.g., 100 or 1000 seconds). However, be aware that your analysis software properly handles wrapped timestamps.

The TICC uses a 32 bit integer to store the count of seconds in its timescale. That variable will overflow after about 68 years, and will retain full 60 picosecond resolution until that happens. But external software used to analyze TICC output files will almost certainly convert the data to floating point format in order to perform the required math functions, and that can result in a problem.

Floating point values in computing require a compromise between range and resolution. That larger the value to the left of the decimal point, the fewer places there can be to the right. The standard 64 bit “IEEE double float” can represent 15.9 decimal digits. To maintain 1 picosecond resolution (12 decimal places), the integer part of a timestamp can’t exceed 8192 seconds without losing resolution in the last decimal place.

Since the TICC actually has about 60 ps rather than 1 ps resolution, the issue takes longer to manifest. Math and experimental results indicate that the first apparent resolution loss occurs at 262,144 seconds (about 3 days) with additional jumps at about 6, 12, 24, etc., days. Here’s an example of that in action; notice the sudden increases in “fuzz” at about 3, 6, and 12 days, just as the math suggests:

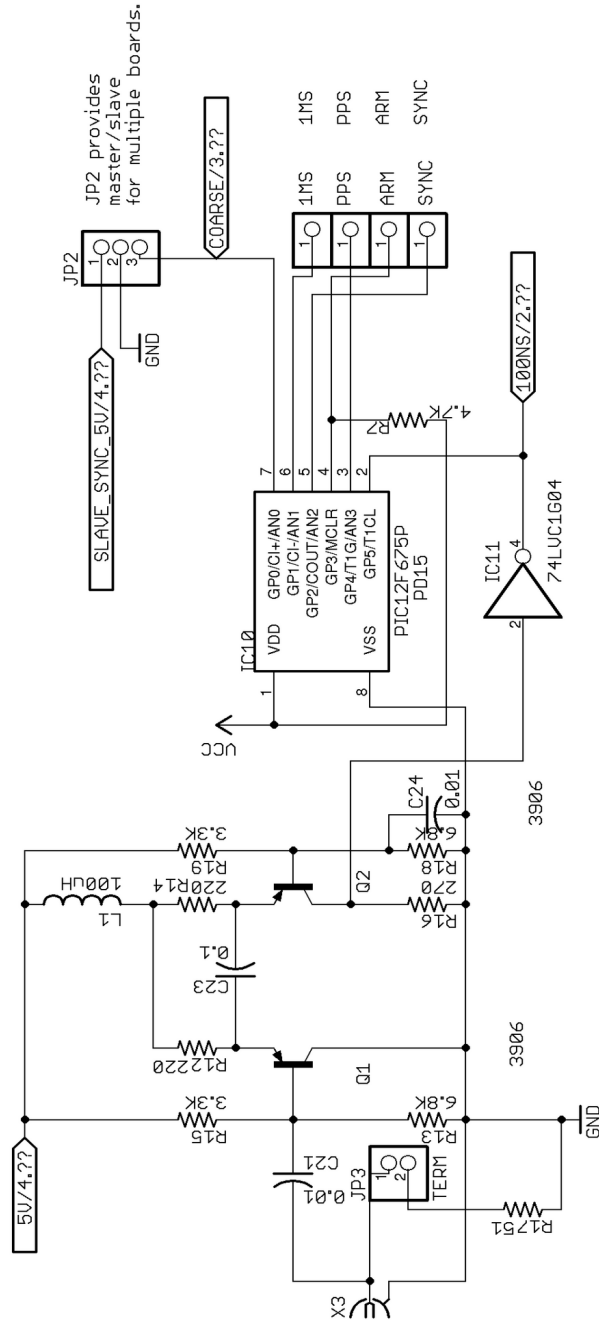


Virtually all analysis software treats frequency or phase data as floating point; it would be very difficult for it to do otherwise. You can’t blame TimeLab for this. But you can take steps to reduce the effect.

The appropriate way to avoid the problem is to wrap timestamps to keep them from becoming too large. In other words, treat the timestamp like an odometer that rolls over at a certain point. If you limit the maximum timestamp value to less than 250,000 seconds, you’ll never lose resolution reading TICC data. As a practical matter, much smaller wraparound values work just as well and the current 202509xx TICC firmware sets the wrap to occur by default at 100 seconds.

Appendix C

TICC Schematics

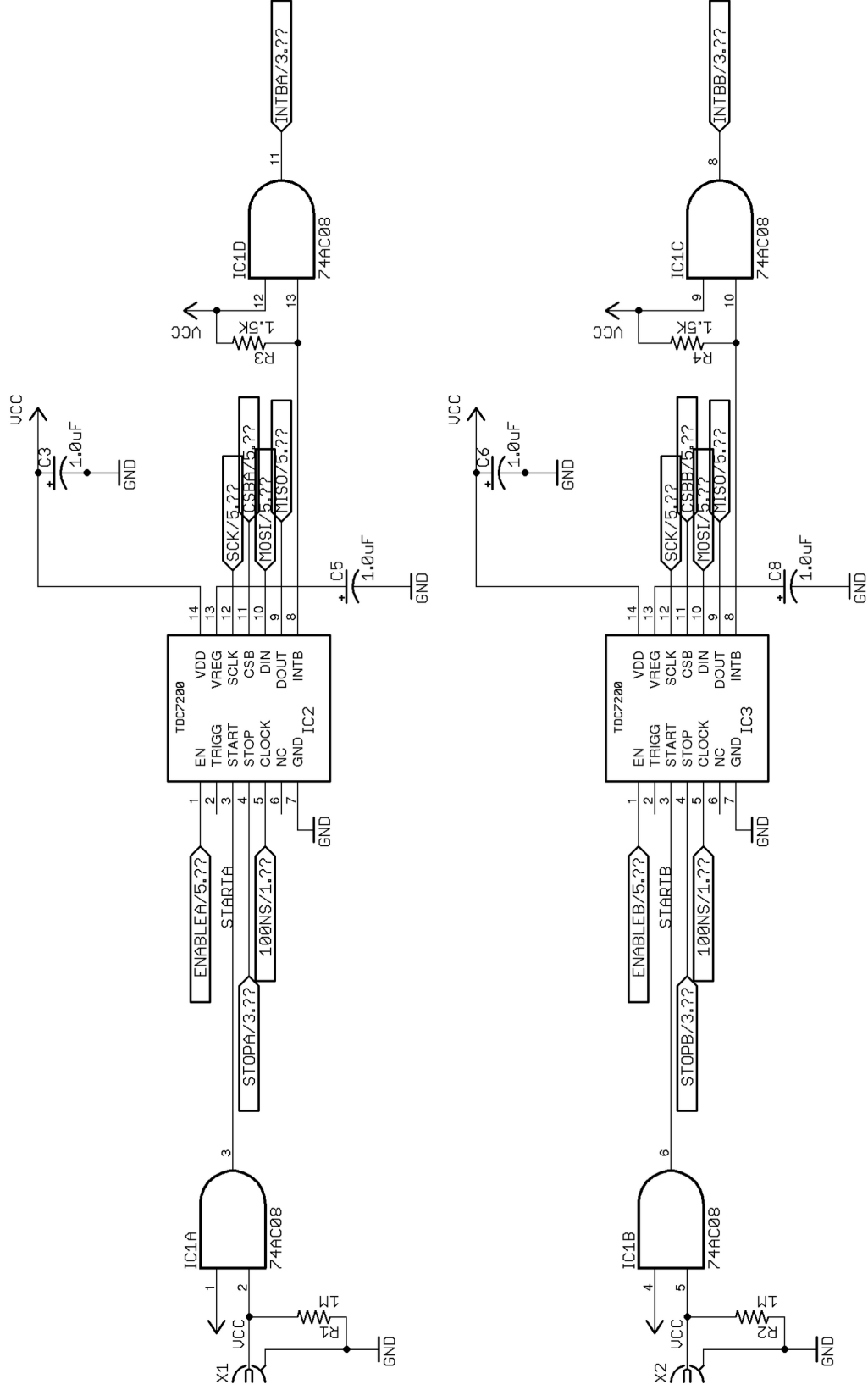


Sine-Square Conv. and PICDIU

TITLE: TICC_rev_d

Document Number: REV:

Date: 11/6/16 8:16 PM Sheet: 1/6



TDC Chips

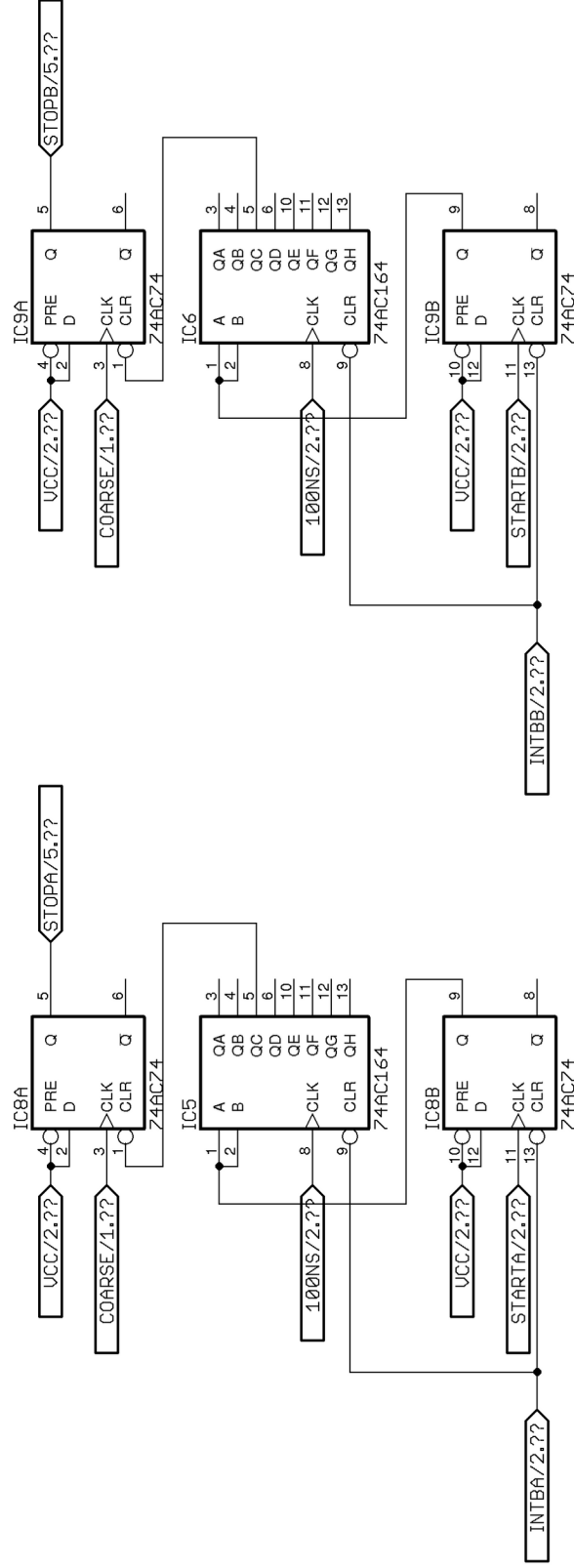
TITLE: TICC_rev_d

Document Number:

REV:

Date: 11/6/16 8:16 PM

Sheet: 2/6



Stop Gate

TITLE: TICC_rev_d

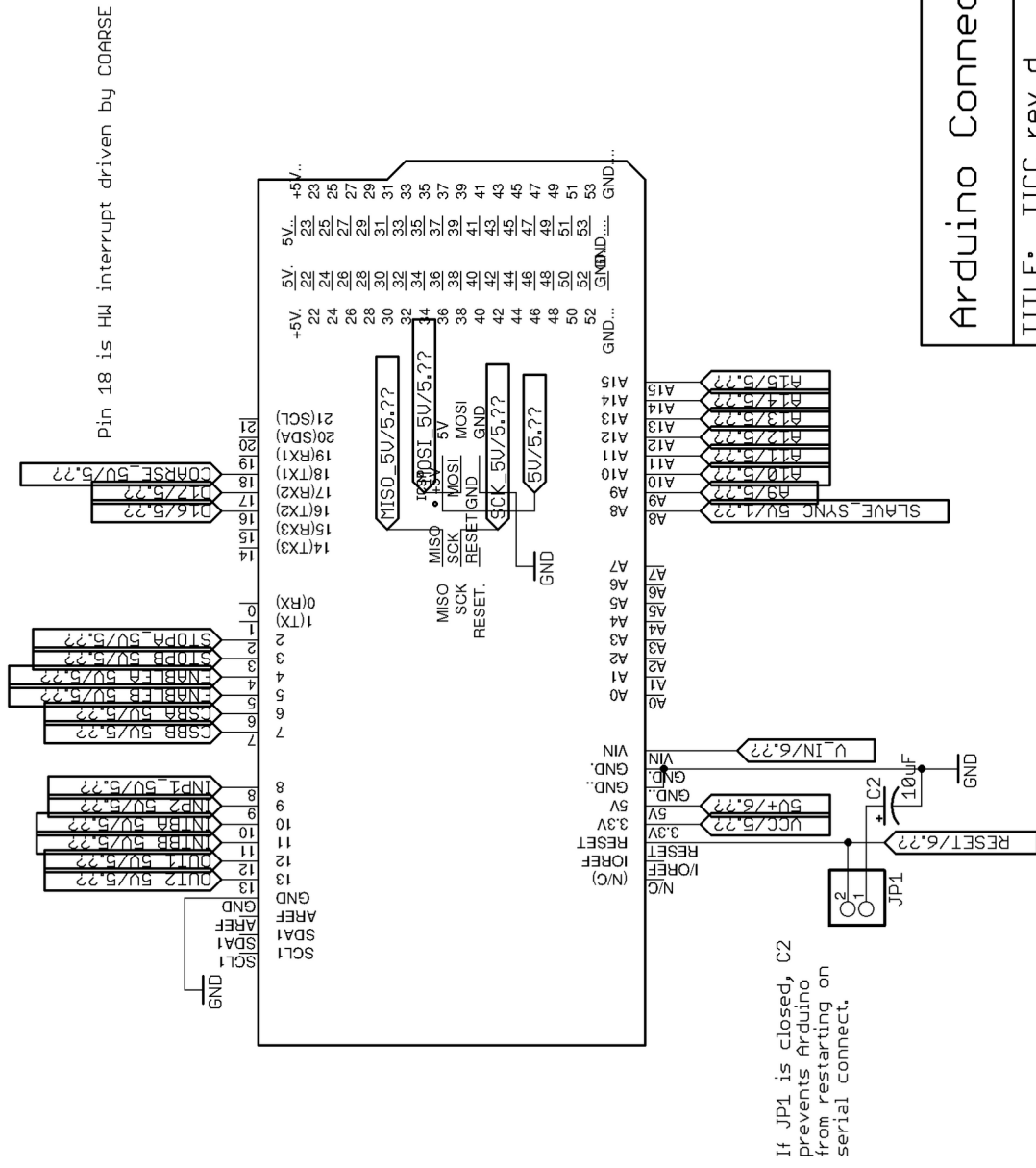
Document Number:

REV:

Date: 11/6/16 8:16 PM

Sheet: 3/6

Pins 2 and 3 are HW interrupts.



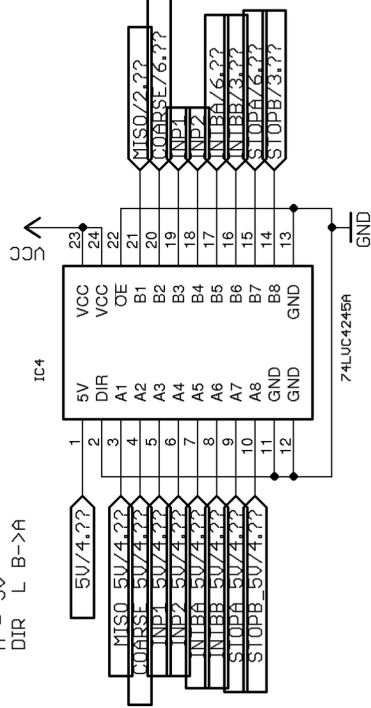
Arduino Connections

TITLE: TICC_rev_d

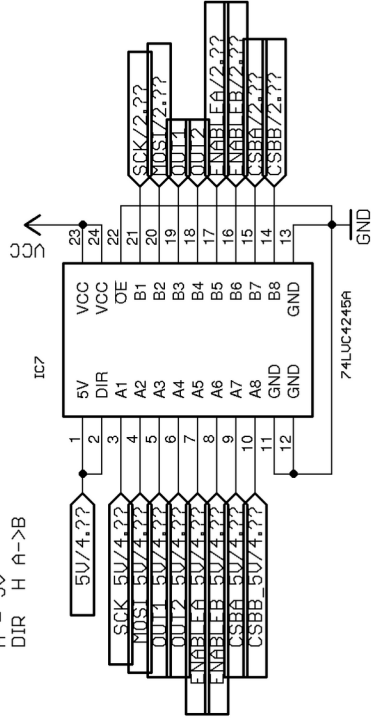
Document Number: REV:

Date: 11/6/16 8:16 PM Sheet: 4/6

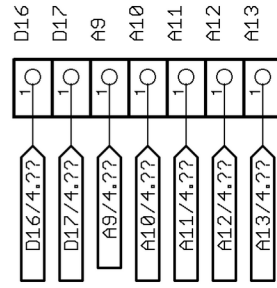
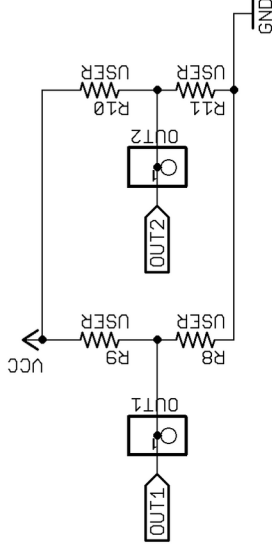
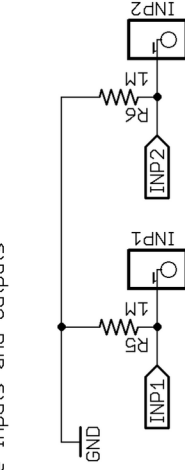
B = 3.3V
A = 5V
DIR L B->A



B = 3.3V
A = 5V
DIR H A->B



Spare inputs and outputs



Level Translation and I/O

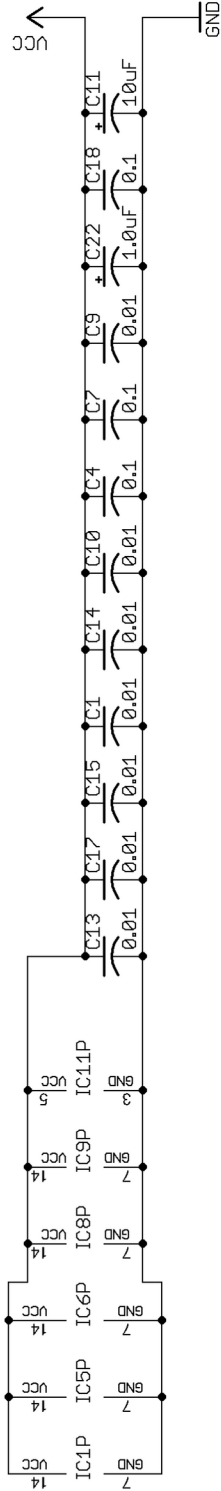
TITLE: TICC_rev_d

Document Number:

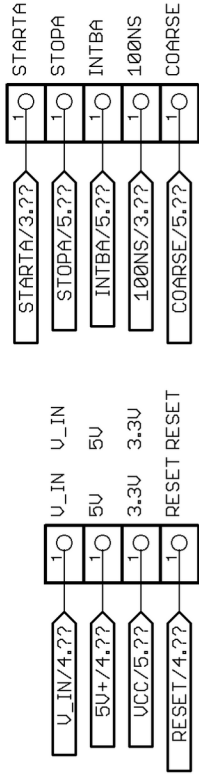
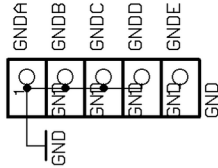
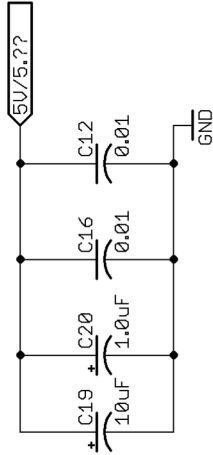
REV:

Date: 11/6/16 8:16 PM

Sheet: 5/6



NOTE: 5V POWER



Power and Test Points

TITLE: TICC_rev_d

Document Number: REV:

Date: 11/6/16 8:16 PM Sheet: 6/6

